

Training an NGram for Java Code

Author: Jeffery Lin

Course: GenAI

Professor: Antonio Mastropaolo

Date: March 6, 2026

Repository: <https://github.com/jl0905/ngram>

1. ABSTRACT

An N-Gram model is essentially a table of all possible permutations of N tokens that are observed in a corpus or training set where we wish to regress, predict, or recommend new sentences/tokens. To utilize the NGram model, we access the conditional probability of the n-th token based on the previous n-1 tokens from our table. For the task of creating an NGram model, we attain a large mass of open source Java methods and we process these methods as training data for our model and also add backoff smoothing. The model is then evaluated twice; on a given test set as well as a random test set I create from unseen corpus data. Evaluation is calculated through the perplexity via model performance on ground truth tokens in the testing set.

2. MSR / DATA

We started the semester by being introduced to how we can mine public GitHub repositories for data to train our models on. My method for MSR extends the minilab python notebook to save the methods directly on my drive so that I can reuse them. We prioritized top starred GitHub repositories while excluding any repos with below 100 stars. Also we read at maximum 20 Java classes per repo that we mine; in addition excluding methods that 1. have below 10 tokens 2. have more than 750 tokens 3. have more than 100 unique tokens. In total I gathered 82,171 methods after 10004 were filtered according to my constraints. The python notebook used to do this can be found here:

<https://drive.google.com/file/d/1tFSuz8h7Qvq5yuv2CBcvHJQ164BabN4V/view?usp=sharing>

3. TRAINING

I chose to implement an NGram class. The constructor for it looks like this:

```

tokenized_input = self.preprocessUnknownTokens(tokenized_input)

self.n = N
self.ngram = self.constructNGram(tokenized_input, N)
self.backoff_ngrams = [self.constructNGram(tokenized_input,
previousN) for previousN in range(N - 1, 0, -1)]

vocab = set()
for method in tokenized_input:
    for token in method:
        vocab.add(token)
self.vocab = list(vocab)

```

We store the N as a constant we can refer to throughout other methods, and we make the actual ngram model which is stored as a dictionary formatted like $(w_{i-N+1}, w_{i-N+2}, \dots): \{w_i: \text{int}, w_i: \text{int}\}$. The keys of `self.ngram` are tuples of context tokens (previous $n - 1$ tokens) and the value of `self.ngram` is a nested dictionary where we find the frequency of the predicted n th tokens.

For our smoothing we implement backoff smoothing, which essentially delegates the task of finding an unseen n -gram to a $(n-1)$ -gram model, and we progressively go down and apply a penalty every time we don't find it. This necessitates `self.backoff_ngrams` which essentially means that (for example) a 10gram will store 9 other dictionaries/models and so on. Add-1 smoothing was attempted at first but this exponentially increased runtime and storage requirements of the model so instead I chose to do backoff smoothing. It is kinda inefficient though, for example when multiple ngram models are made we repeat a lot of work in making the backoff ngrams.

Some constants in the code I chose include string literals "<START>", "<END>", "<UNK>" and these are the only abstract tokenizations I chose to implement. I choose another arbitrary constant with regard to the backoff smoothing hyperparameter: the penalty we apply in backoff smoothing is a factor of 0.8 every time the current ngram is not identified. So if $P(C | A, B)$ is not in the model then we delegate to $0.8 * P(C | B)$ and then to $0.8 * 0.8 * P(C | \text{""})$. I didn't tune the backoff smoothing at all but I did tune the N according to the instructions for the assignment and I found that an N of 3 is the best.

4. RESULTS

When running the `benchmark()` function I created I found the following results:

The perplexity for a 3-gram model trained from 15000 entries of data is: 25.879293101550605
The perplexity for a 3-gram model trained from 25000 entries of data is: 26.799387630874385
The perplexity for a 3-gram model trained from 35000 entries of data is: 26.799036210784617
The perplexity for a 5-gram model trained from 15000 entries of data is: 67.30846274563129
The perplexity for a 5-gram model trained from 25000 entries of data is: 70.91726735310597
The perplexity for a 5-gram model trained from 35000 entries of data is: 69.93172688556855
The perplexity for a 7-gram model trained from 15000 entries of data is: 150.16779569548515
The perplexity for a 7-gram model trained from 25000 entries of data is: 163.94903340554254
The perplexity for a 7-gram model trained from 35000 entries of data is: 165.2176908598656

This is somewhat disconcerting because we are supposed to have less perplexity as more data is fed to the model, but in my case the perplexity has increased which could be a case of error in the way the data is organized or the perplexity is calculated. My theory is-- that since, in my case and probably others too, my training data is sorted by repo and the repos are ordered by the number of stars they have-- the first methods in the list comes from Java codebases that are more generalizable to the language syntactically and semantically. For example the first two repos in `methods.txt` are `Snailclimb/JavaGuide` and `krahets/hello-algo` which probably hold much more semantically generalizable code and do not have many unseen tokens or extremely specific methods. Subsequently the first methods in `all_methods` come from these and are intrinsically better datum to train on than the methods that come later in `all_methods`.

The method for the above results is `benchmark()`, however it is unused in the “production” version of my code since I just ran it once and from the results decided to use a 3-gram model trained from the first 25000 methods of the corpus for further inference. I chose it slightly randomly because the three 3-grams are very close in perplexity on validation and I thought it might be less overfit.

For the output format I got rid of `predToken` since we aren’t doing `argmax`. Finally, the perplexity on the class test set and personal test set are reasonable— my model gets 18.389 on the test set given to us and 25.948 on the test set created by myself which I consider a mild success since they are both lower than the validation score.